AdForge AI: AI-Powered Advertisement Copy Generator

Project Description:

AdForge AI harnesses **Generative AI** to provide intelligent multi-platform advertisement copy generation, enabling marketers and businesses to create engaging, platform-specific marketing content quickly and efficiently. The platform includes an **Ad Copy Generator** that produces multiple unique ad variations per request, a **Slogan Generator** that creates impactful brand slogans, a MultiPlatform Adaptation module that optimizes content for **Google Ads, Instagram Ads**, and **Facebook Ads**, and a **Refine Copy feature** that improves existing advertisement copy based on user feedback.

Utilizing **Groq's API** with the **Qwen/Qwen3-32B Versatile** model, **AdForge AI** processes structured product briefs to generate fast, high-quality, and conversion-focused advertisement content. Built with **Flask** on the backend and **HTML, CSS, and JavaScript** on the frontend, the platform delivers a seamless user experience. With secure API key management through **.env** and responsible input handling, **AdForge AI** empowers marketers, creators, and businesses to craft effective advertising campaigns with confidence.

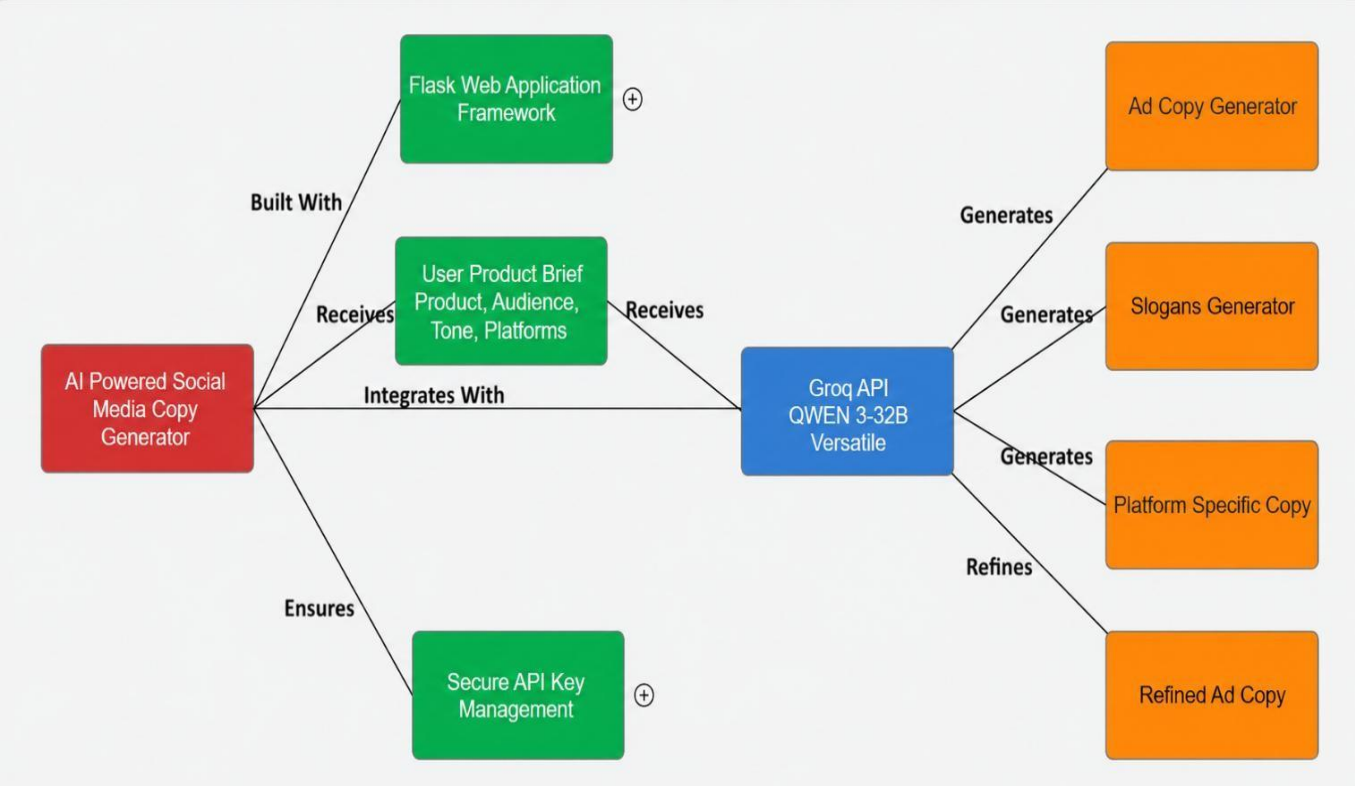
Scenarios

Scenario 1: A user enters product details, selects **Google Ads**, **Instagram Ads**, and **Facebook Ads** with a **Playful** tone. AdForge AI generates platform-specific ad copies, marketing slogans, and relevant hashtags. The content is optimized for each selected platform.

Scenario 2: A user promoting a B2B SaaS product selects **Google Ads** and **Facebook Ads** with a **Professional** tone. The platform generates keyword-optimized ad copy, persuasive CTAs, and professional marketing content. The output is tailored to attract business customers.

Scenario 3: A user launches a flash sale and selects all platforms with an Urgent tone. AdForge AI creates urgency-driven advertisements, engaging captions, and promotional copy. The content emphasizes limited-time offers to maximize engagement and conversions.

Technical Architecture:



Description: AdForge AI utilizes a modular three-tier architecture to deliver fast, AI-powered advertisement copy generation. The frontend provides a responsive user interface built with **HTML, CSS, and JavaScript**, while the **Flask-based backend** manages user input validation, prompt engineering, and request processing. It integrates with the **Groq API** using the **Qwen/Qwen3-32B Versatile** model to generate platform-specific advertisement copy, slogans, and refined marketing content, with secure API key management via **.env** and deployment through **Flask** and **Ngrok** for seamless public access.

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Groq API Familiarity:** Groq API Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide
- **Ngrok for Public Deployment:** Ngrok Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a **Groq API** key and configure it securely using environment variables (.env).
- **Activity 1.2:** Research and select the **Qwen/Qwen3-32B Versatile** model for AI-powered advertisement copy generation.
- **Activity 1.3:** Design the system architecture, defining interactions between the frontend, Flask backend, and Groq API.
- **Activity 1.4:** Set up the development environment by installing Flask, required Python libraries, and project dependencies.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the **Ad Copy Generator**, **Slogan Generator**, **Multi-Platform Adaptation**, and **Refine Copy** modules.
- **Activity 2.2:** Implement the Flask backend to process user inputs, construct AI prompts, and communicate with the Groq API.

Milestone 3: App.py Development

- **Activity 3.1:** Develop the main application logic in **app.py**, define application routes, and integrate AI-generated responses.
- **Activity 3.2:** Implement input validation, error handling, and secure API request management.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, for advertisement generation.
- **Activity 4.2:** Create dynamic templates with Flask's render_template to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Test all modules to ensure accurate AI responses, platform-specific formatting, and smooth user interactions.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 6: Conclusion

- **Activity 6.1:** Evaluate the application's performance, usability, and AI-generated output quality.
- **Activity 6.2:** Document the project outcomes, limitations, and potential future enhancements for multi-platform advertisement generation.

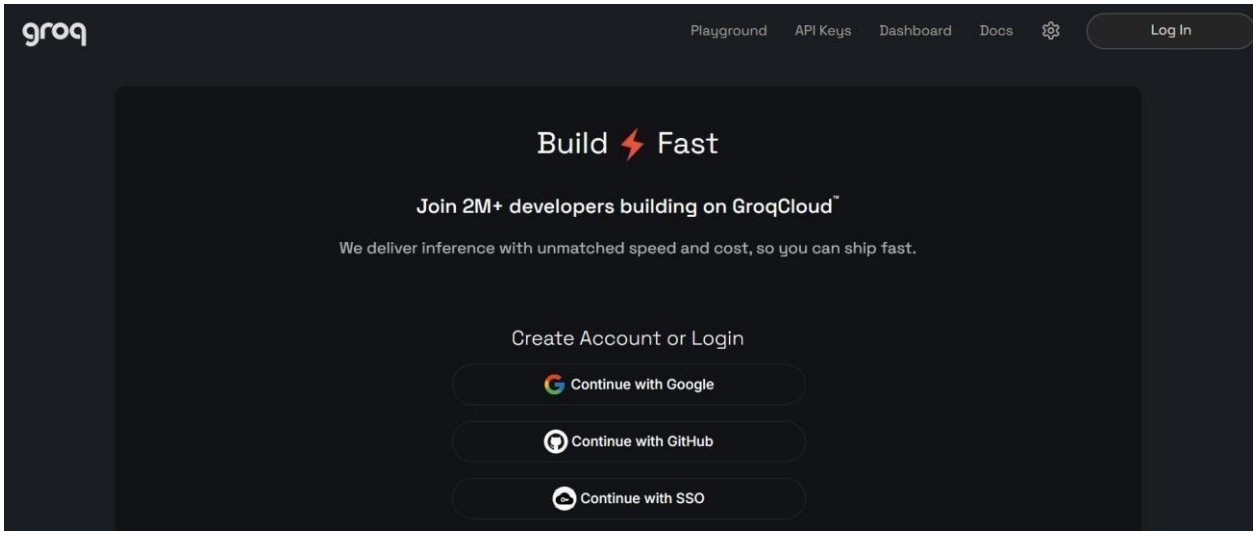
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate **Generative AI model** from **Groq** for advertisement copy generation. This involves researching the capabilities and performance of various models offered by Groq, ensuring that the selected model aligns with the application's objective of generating high-quality advertisement copy, marketing slogans and platform-specific content for **Google Ads**, **Instagram Ads**, and **Facebook Ads.**

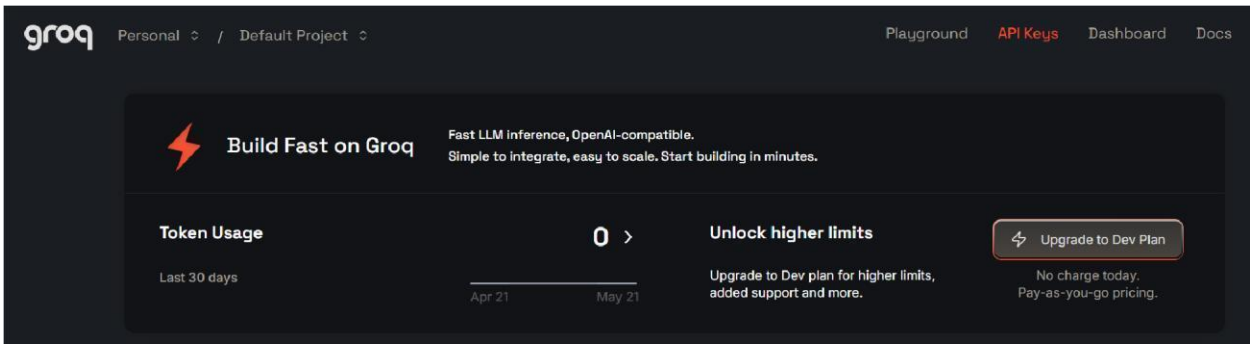
Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

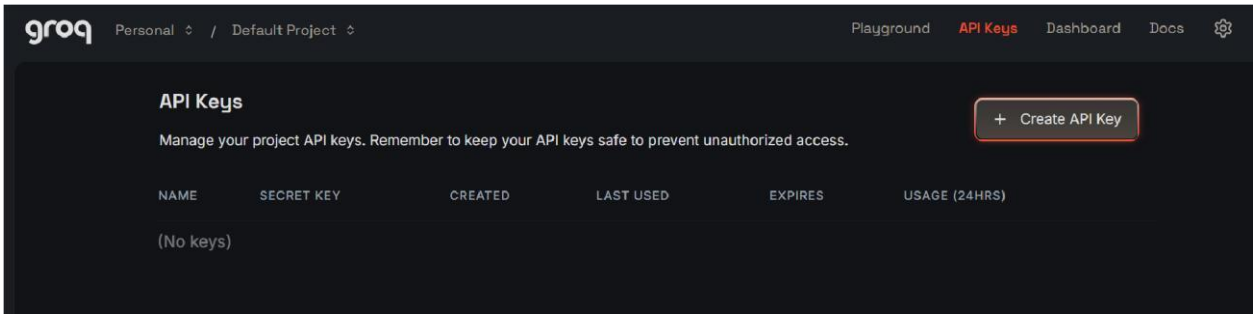
- **Step 1 — Create a Groq Account**: Visit https://console.groq.com and sign up for a free account using your Google account or email address.



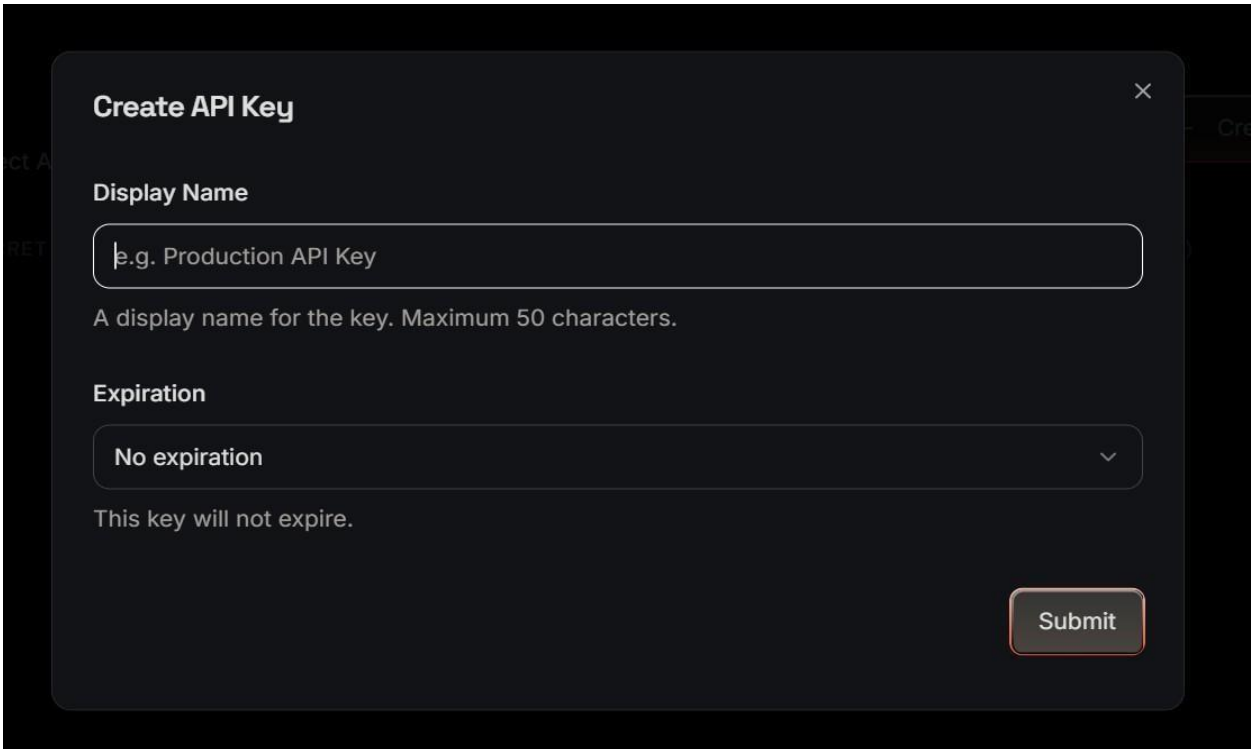
- **Step 2 — Navigate to API Keys**: After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the API Keys page.



- **Step 3 — Create a New API Key:** Click the "Create API Key" button. Give your key a descriptive name (e.g., captionai-key) so it is easy to identify later.



- **Step 4 — API Key:** Now give a "Display Name" and "Expiration" is optional, and Click on "Submit". The you will get your API key, Copy it immediately and store it in a safe place you will not be able to view it again after closing the dialog.



- **Step 5 — Add the Key to Your Project:** Create a .env file in the root directory of your AdForge AI project (if it does not already exist) and store the generated Groq API key securely.

GROQ_API_KEY=your_copied_api_key_here.

- **Step 6 — Verify the Key is Loaded:** The app.py file loads this key automatically at startup using python-dotenv: from dotenv import load_dotenv load_dotenv()

client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

- **Important** — Never Commit Your API Key: Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository: .env

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to research to select a best model for the project:

- **Understand the Project Requirements:** Analyse the application's requirements, focusing on generating advertisement copy, marketing slogans, and platform-specific content for Google Ads, Instagram Ads, and Facebook Ads across multiple writing tones.
- **Explore Groq's Model Documentation:** Review the models available through the Groq platform, examining their capabilities, context window, inference speed, and suitability for marketing and advertisement content generation.
- **Evaluate Model Performance:** Compare different models based on content quality, response speed, contextual understanding, creativity, and their ability to generate persuasive, platform-optimized advertisements.
- **Select the Optimal Model:** Choose Qwen/Qwen3-32B-versatile for its excellent balance of speed, creativity, and contextual accuracy. Configure the model with a temperature of 0.8 and max_tokens of 2048 to generate high-quality, engaging, and conversion-focused advertisement content.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the **HTML, CSS, and JavaScript frontend**, **Flask backend**, and **Groq API** integration.
- **Detail Frontend Functionality:** Define how users interact with the application by entering **product details**, selecting one or more platforms (**Google Ads, Instagram Ads, Facebook Ads**), choosing a **tone** (Professional, Playful, Urgent, Casual, etc.), and submitting the request for AI-generated advertisement content.

- **Outline Backend Responsibilities:** Specify how the **Flask backend** receives POST requests, validates and processes user inputs, constructs structured prompts, communicates with the **Groq API**, and manages secure API interactions.
- **Describe AI Integration Points:** Explain how the backend generates prompts based on the user's **product information, target audience, selected platforms, and tone**, sends them to the **LLaMA 3.3-70B Versatile** model through the **Groq API**, processes the AI response, and returns structured outputs including **advertisement copy, marketing slogans, platform-specific content, and refined ad copy** to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv

D:\projects>"d:\projects\venv\Scripts\activate"

(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

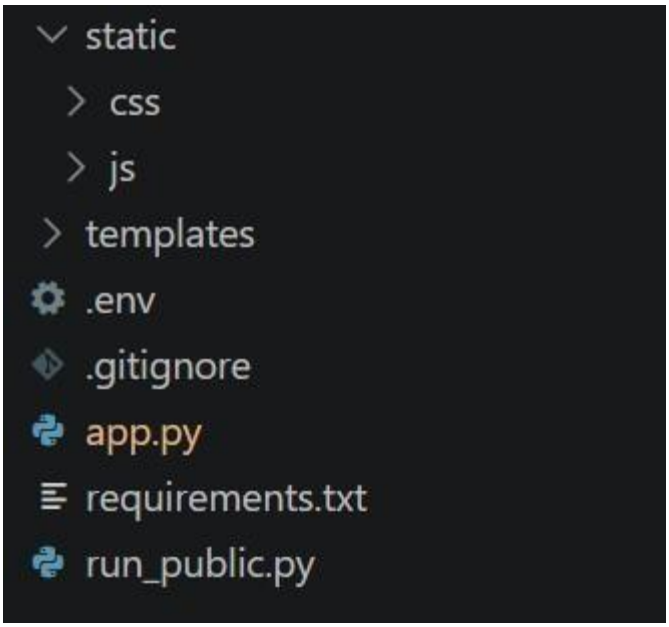
```
(myenv) D:\projects> pip install groq==0.11.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:
  `GROQ_API_KEY=your_groq_api_key_here`

- **Set Up Application Structure:** Organize the AdForge AI project by creating folders for **templates**, **static** (CSS, JavaScript, and images), and application files such as **app.py**, **requirements.txt**, **.env**, and **run_public.py**. Design the project structure to support modules for **Ad Copy Generation**, **Slogan Generation**, **Multi-Platform Adaptation**, and **Refine Copy**.



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the core AI-powered advertisement generation features using a single structured prompt to generate complete marketing content.

- **Ad Copy Generator:** Generates multiple unique, platform-specific advertisement copies tailored to the selected product, target audience, platform, and writing tone**.**
- **Slogan Generator:** Creates three memorable and impactful marketing slogans that align with the product's identity and branding objectives.
- **Multi-Platform Adaptation:** Produces optimized advertisement content for Google Ads, Instagram Ads, and Facebook Ads, following each platform's recommended format and style.
- **Refine Copy:** Enhances existing advertisement copy based on user feedback by improving clarity, engagement, tone, and overall marketing effectiveness.

Activity 2.2: Implement the Flask Backend Define Routes in Flask:

- Set up routes in app.py for the home page and content generation endpoint.

Process User Input:

- Create a form in **index.html** where users can enter product details, target audience, select one or more advertising platforms, and choose a preferred writing tone.
- Use Flask's **request.get_json()** or **request.form** to retrieve the submitted data from the frontend.
- Validate that all required fields are provided and sanitize user input before processing the AI request.

Integrate API Calls:

- Within the **/generate** route, build a structured prompt using the user's product details, selected platforms, target audience, and tone.
- Send the prompt to the **Groq API** using **client.chat.completions.create()** with the **Qwen/Qwen3-32B-versatile** model.
- Process the AI response and return structured outputs containing **advertisement copies**, **marketing slogans**, **platform-specific content**, and **refined advertisement copy** to the frontend.

Milestone 3: App.py Development

Milestone 3 focuses on developing the core functionality of the **app.py** file, which serves as the backbone of the **AdForge AI** application. In this milestone, Flask routes are implemented to process user requests, generate structured AI prompts, communicate with the Groq API, and return advertisement content. Input validation, error handling, and response processing are also implemented to ensure reliable application performance.

Activity 3.1: Writing the Main Application Logic in app.py Define the Core Routes in app.py:

- Establish routes for AdForge AI's two endpoints:
  - **/** — Home page route that renders the main index.html interface

```
@app.route("/")  def
index():
    return render_template("index.html")
```

  - **/generate** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

```
@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json()


    product_info = data.get("product_info", "").strip()      platform
= data.get("platform", "instagram").lower()      tone =
data.get("tone", "professional").lower()
```

```
        return jsonify({"error": "Product information is
required."}), 400     if len(product_info) > 2000:          return
jsonify({"error": "Input too long. Please keep it under
2000 characters."}), 400
try:
        prompt = build_prompt(product_info, platform, tone)


        chat_completion = client.chat.completions.create(
messages=[
                {
                    "role": "system",


                    "content": "You are a social media content
expert. Always respond with valid JSON  only, no markdown
formatting."

                },


                {
                    "role": "user",
                    "content": prompt
                }
],
            model="llama-3.3-70b-versatile",
            temperature=0.8,
max_tokens=2048,
        )


        response_text = chat_completion.choices[0].message.content
result = extract_json(response_text)
```

```
        # Validate structure           if not all(k in result for k
in ["captions", "hashtags",
"ctas"]):
            raise ValueError("Invalid response structure from AI")
return jsonify({             "success": True,                        if
"captions": result["captions"],


            "hashtags": result["hashtags"],
            "ctas": result["ctas"],
            "platform": platform,
not product_info:
```

```
            "tone": tone

        })      except ValueError
as e:
        import traceback
        traceback.print_exc()          return jsonify({"error":
f"Failed to parse AI response:
{str(e)}"}), 500      except
Exception as e:
import traceback
traceback.print_exc()
        return jsonify({"error": f"Generation failed: {str(e)}"}), 500
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads product_info, platform, and tone from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if product_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- **TONE_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.

```
TONE_DESCRIPTIONS = {
    "professional": "formal, authoritative, and business-oriented. Use clear, concise language that  conveys
expertise and credibility.",
    "casual": "friendly, conversational, and relatable. Use informal language, emojis, and a warm tone
```

```
that feels personal.",
    "promotional": "exciting, persuasive, and action-driven. Use power words, urgency, and compelling offers
to drive engagement."

}
```

- **PLATFORM_TIPS** dictionary: maps instagram and linkedin to platform-specific content guidelines (character limits, style expectations).

```
PLATFORM_TIPS = {
    "instagram": "optimized for Instagram — visually descriptive, emotionally engaging, with strong visual
storytelling. Max 2200 characters.",
    "linkedin": "optimized for LinkedIn — professional insights, value-driven, thought leadership style. Max
3000 characters."
}
```

- **build_prompt()**: assembles a structured prompt string from the product info, selected platform tip, and tone description, instructing the model to return strictly valid JSON.

```
def build_prompt(product_info: str, platform: str, tone: str) -> str:
    tone_desc = TONE_DESCRIPTIONS.get(tone, "engaging")      platform_tip
= PLATFORM_TIPS.get(platform, "social media")



                                                                    return
f"""You are an expert social media content strategist and copywriter.
```

Integrate the Groq AI Responses in Each Function:

- Call client.chat.completions.create() with a system message enforcing JSON-only responses and a user message containing the assembled prompt.

- **extract_json()**: a robust parser that first attempts direct json.loads(), then strips markdown code fences (```json blocks), and finally uses a regex to locate a raw JSON object ensuring reliable parsing across model response variations.

```
def extract_json(text: str) -> dict:

    """Extract JSON from LLM response, handling markdown code blocks."""
# Try direct parse first      try:

        return json.loads(text)



    except json.JSONDecodeError:
pass
    # Try extracting from markdown code block      match = re.search(r'```(?:json)?\s*([\s\S]*?)```',
text)      if match:
try:
            return json.loads(match.group(1).strip())          except
json.JSONDecodeError:              pass      match =
re.search(r'\{[\s\S]*\}', text)      if match:          try:

            return json.loads(match.group(0))          except
json.JSONDecodeError:              pass      raise
ValueError("Could not extract valid JSON from response")
```

- Validate that the parsed result contains all three required keys — captions, hashtags, and ctas — before returning it to the frontend.
- **Home route:** / → renders index.html/generate (POST) → accepts JSON, returns {success,captions, hashtags, ctas, platform, tone}

Milestone 4: Frontend Development

Milestone 4 focuses on developing a modern, responsive, and user-friendly interface for **AdForge AI**. This involves designing an intuitive layout using **HTML**, **CSS**, and **JavaScript** to simplify

-

advertisement generation and dynamically display AI-generated ad copies, slogans, and platformspecific marketing content.

Activity 4.1: Designing and Developing the User Interface Set Up the Base HTML Structure:

Develop the main HTML file (**index.html**) as the application's single-page interface, including a header with the **AdForge AI** logo and tagline, an advertisement input form, and a results section.

- Use semantic HTML elements to organize the page structure and improve accessibility.
- Integrate **Font Awesome** icons and **Google Fonts (Inter)** to create a modern and professional user interface.

Design a Responsive Layout Using CSS:

- Create a stylesheet (**static/css/style.css**) featuring a modern, responsive design with attractive colors, cards, and smooth animations.
- Use **Flexbox** and **CSS Grid** to organize input forms and generated advertisement results into a clean and structured layout.
- Add media queries to ensure compatibility across desktop, tablet, and mobile devices.

Create Separate UI Components for Each Output Type:

- **Advertisement Cards:** Display each generated advertisement copy inside individual cards with a one-click Copy button.
- **Slogan Section**: Present AI-generated marketing slogans in a dedicated card for easy reading and copying.
- **Refine Copy Panel:** Provide a dedicated section where users can view improved advertisement copy generated from their feedback.
- **Platform-Specific Content:** Display advertisement content separately for Google Ads, Instagram Ads, and Facebook Ads, allowing users to compare platform-optimized outputs.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript Handle Form Submission Asynchronously:

- Attach a submit event listener that sends user inputs asynchronously to the **/generate** endpoint using the **Fetch API**.
- Display a loading animation while the AI processes the advertisement request and temporarily disable the **Generate** button.

-

Render Results Dynamically:

- Implement a **renderResults()** function in **main.js** to dynamically display advertisement copies, marketing slogans, platform-specific advertisements, and refined content.
- Automatically scroll to the results section after successful content generation.

Restore UI State After Generation:

Re-enable the **Generate** button once processing is complete and restore the original button text and icon

- Display appropriate success or error messages to provide clear feedback to the user.

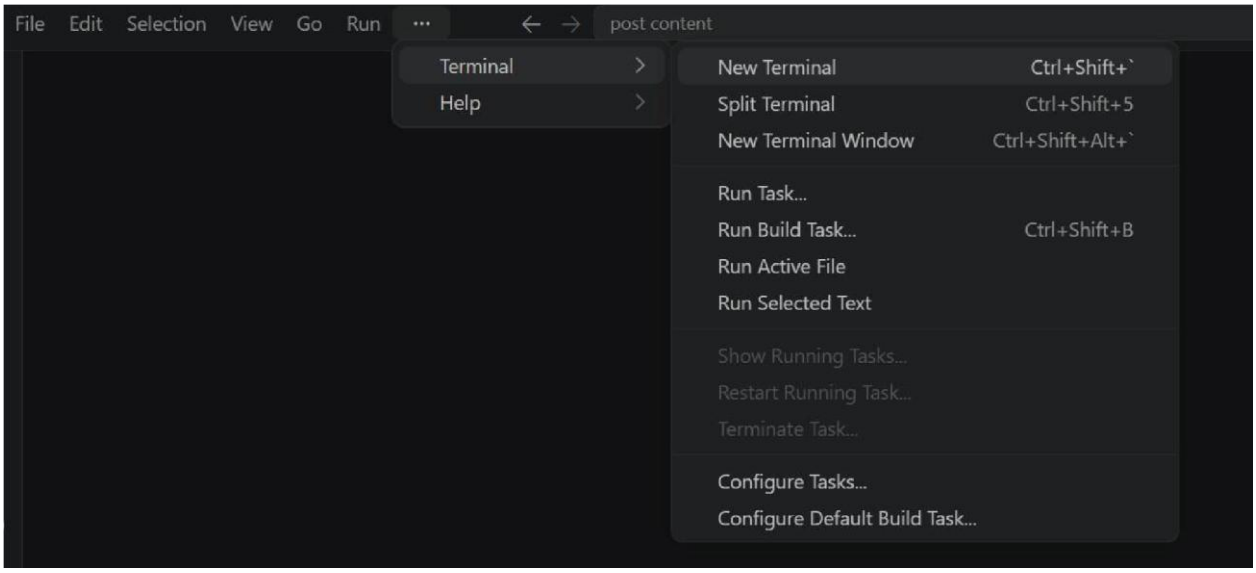
Milestone 5: Deployment

In **Milestone 5**, the focus is on deploying the **AdForge AI** application by first running it locally to verify its functionality and then making it publicly accessible using **Ngrok**. This milestone involves configuring the Flask server, managing project dependencies, securing API credentials, and ensuring seamless communication with the Groq API. The deployment process enables users to access the application through a secure public URL and generate AI-powered advertisement content from any device.

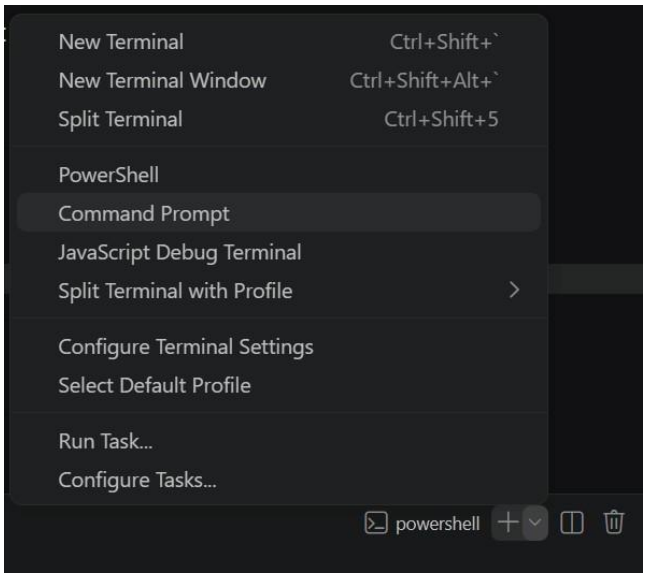
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal

- 

- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv

D:\projects>"d:\projects\venv\Scripts\activate"

(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:

  GROQ_API_KEY=your_groq_api_key_here

Activity 5.2: Local Testing and Verification Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app.

```
(venv) D:\projectsSB\AI in healthcare\startup-validator>python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This                                  ver. Do not use it in a production deployment.
d.                  Follow link (ctrl + click)
 * Running on http://127.0.0.1:5050
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 442-110-195
```

- Interact with **AdForge AI** by entering product details, selecting one or more advertising platforms (**Google Ads**, **Instagram Ads**, **Facebook Ads**), and choosing a writing tone (**Professional**, **Playful**, **Urgent**, **Casual**, etc.). Verify that the application correctly generates advertisement copies, marketing slogans, platform-specific content, and refined advertisement copy.

Activity 5.3: Public Deployment via Ngrok

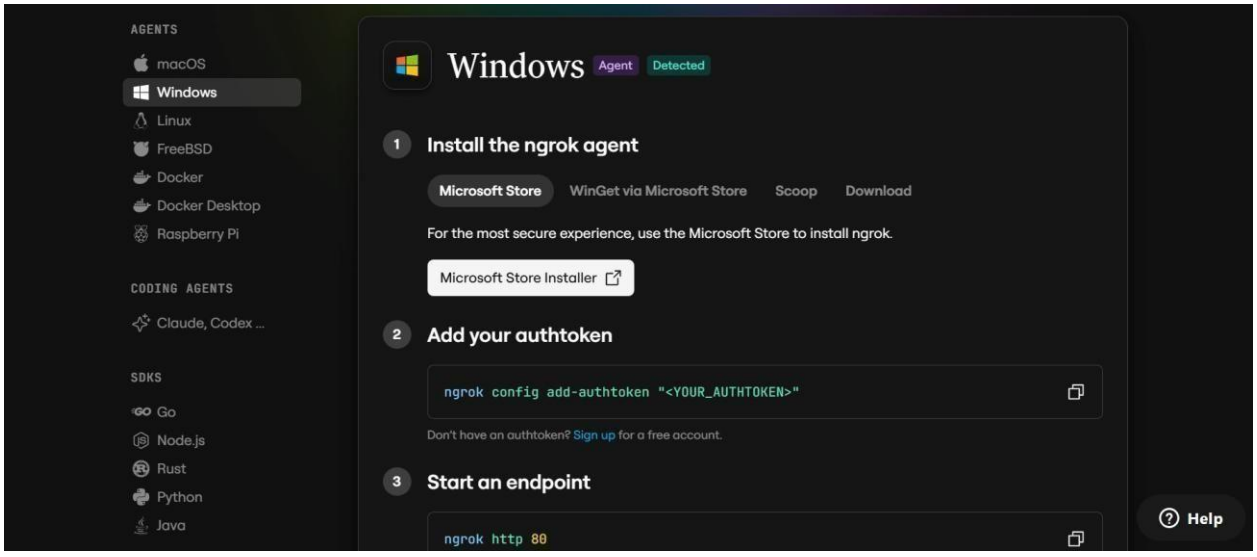
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from https://ngrok.com/download.
- Click on "Microsoft Store Installer".



Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:

```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

```
========================================================= YOUR ADFORGE
AI IS NOW LIVE!!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=========================================================
```

- Share this URL so users can access **AdForge AI** from any web browser and generate AIpowered advertisement content.

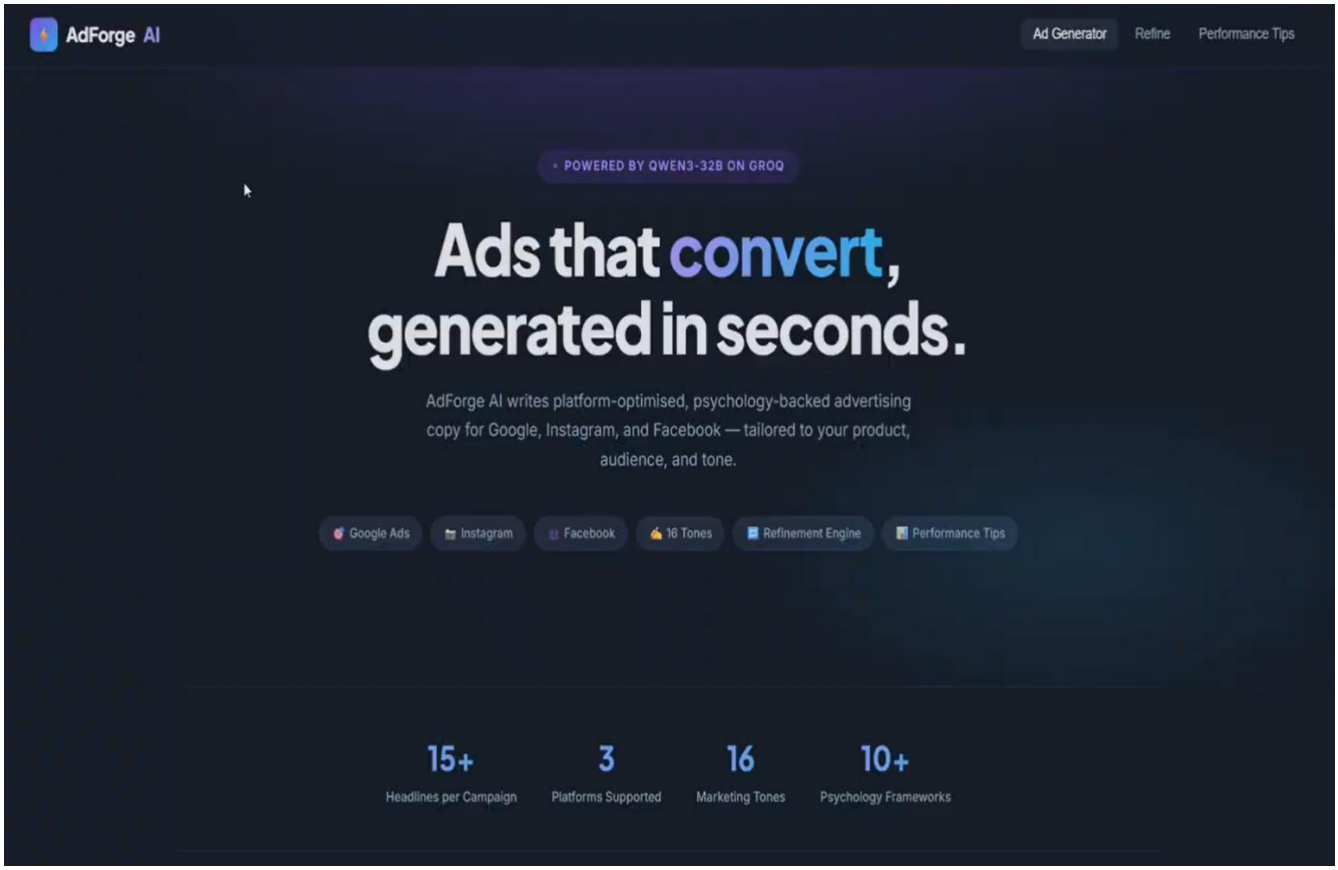
Important Notes:

- **debug=False** is required in run_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.

- Ngrok free tier sessions expire after a few hours of inactivity. Restart run_public.py to get a new URL when this happens.

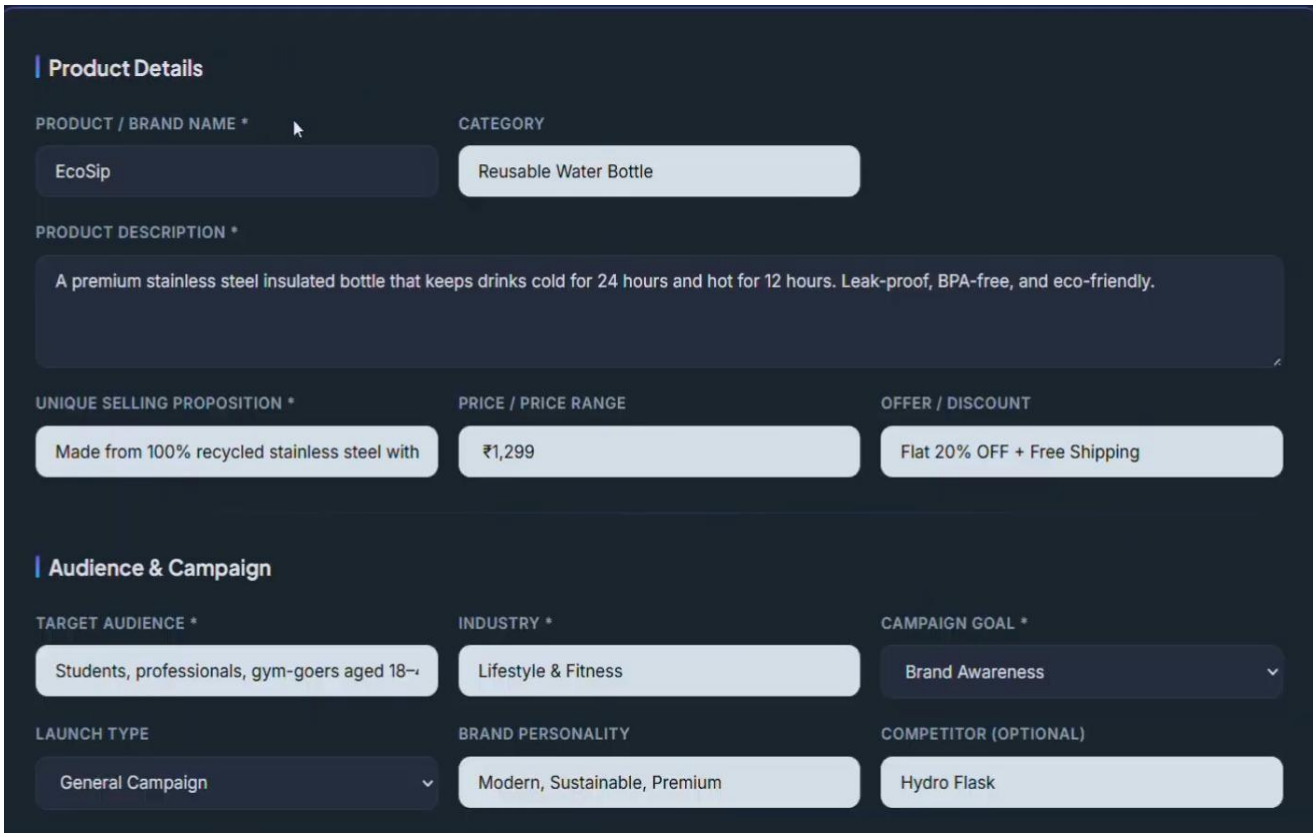
Exploring the Website's Web Pages:

Home / Generator Page:



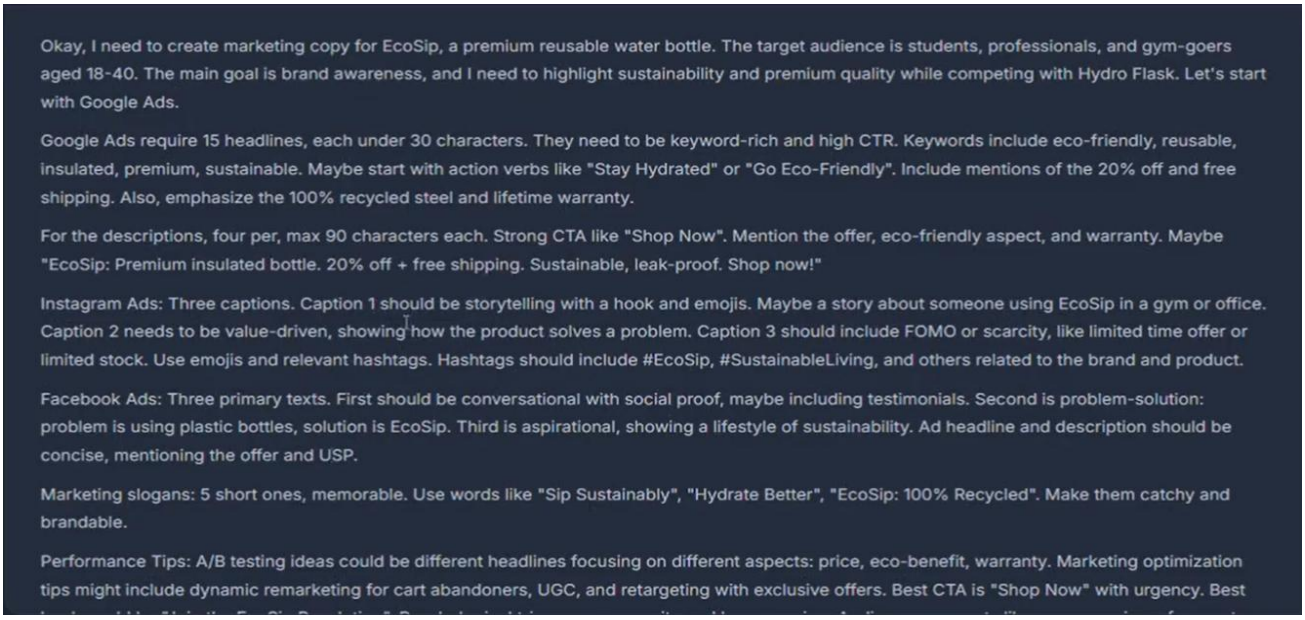
Description: The single-page interface of **AdForge AI** serves as both the landing page and the advertisement generation platform. It features a modern, responsive design with an attractive user interface, a header displaying the **AdForge AI** logo and tagline, and a streamlined layout for content creation. The page includes the advertisement input form, platform selection, tone selector, and AIgenerated results section, providing users with a seamless experience for creating marketing content.

Input Section:



Description: The input section contains the main advertisement generation form where users enter product or service details, specify the target audience, select one or more advertising platforms (**Google Ads**, **Instagram Ads**, or **Facebook Ads**), and choose a preferred writing tone (**Professional**, **Playful**, **Urgent**, **Casual**, etc.). After submitting the request, the **Generate** button displays a loading animation while the AI processes the input and generates advertisement content.

Results Section:



Description: Once generation is complete, the results section is displayed dynamically, presenting AI-generated advertisement content in a well-organized layout. It includes multiple **advertisement copies**, **marketing slogans**, **platform-specific advertisements** for the selected platforms, and a **Refined Copy** section containing improved advertisement content based on user feedback. Each generated result includes a **Copy** button for easy clipboard access, and the page automatically scrolls to the results section after successful content generation.

Conclusion:

AdForge AI is a Generative AI-powered web application developed using **Flask** that simplifies the creation of high-quality, platform-specific advertisement content for digital marketing. By leveraging **Groq's high-speed inference API** with the **Qwen/Qwen3-32B Versatile** model, the application generates engaging advertisement copies, marketing slogans, platform-specific content for **Google Ads**, **Instagram Ads**, and **Facebook Ads**, along with refined advertisement copy based on user feedback. The project successfully integrates AI model selection, prompt engineering, backend development, frontend design, and public deployment using **Ngrok**, demonstrating how Generative AI can enhance marketing productivity and creativity.

Looking ahead, **AdForge AI** offers significant opportunities for future enhancement, including support for additional advertising platforms such as **LinkedIn Ads** and **X (Twitter) Ads**, multilingual advertisement generation, AI-powered image and banner recommendations, campaign performance optimization, and user authentication for saving advertisement history. By continuously improving prompt engineering, expanding platform support, and enhancing the user experience, **AdForge AI** has the potential to evolve into a comprehensive AI-powered advertising assistant for marketers, content creators, startups, and businesses.